

Nettoyage des données — suite

Analyse des données — Séance 3, suite

Stefania Marcassa

CY Cergy Paris Université — L3 Économie

Où nous en sommes

La semaine dernière, nous avons vu comment **trouver** les valeurs manquantes, et distingué leurs trois origines possibles.

Complètement au hasard	l'absence ne dépend de rien
Au hasard, conditionnellement	l'absence dépend de variables observées
Non au hasard	l'absence dépend de la valeur elle-même

Aujourd'hui : reconnaître le troisième cas, puis **décider quoi faire** — et documenter la décision.

Le fil de la séance

Le nettoyage n'est pas une opération technique. C'est une suite de décisions, dont chacune modifie le résultat.

Reconnaître le troisième cas

```
df["nr"] = df["SALAIRE_NET"] == 999999  
100 * df.groupby("DIPLOME")["nr"].mean()
```

DIPLOME	1	2	3	4	5	6	7	9
	aucun	BEPC	CAP	bac	+2	+3/4	+5	n.r.
Non-réponse (%)	3,3	4,4	4,9	6,3	8,0	7,7	10,6	5,9

Trois fois plus de non-réponse en haut de l'échelle des diplômes qu'en bas. La non-réponse dépend d'une variable fortement liée au salaire, ce qui rend l'hypothèse de hasard intenable.

La dernière colonne n'est pas un niveau de diplôme. C'est la non-réponse au diplôme, et ses 5,9 % ne se lisent pas sur le gradient. Une modalité « non-réponse » qui apparaît dans un profil est une erreur de nettoyage, pas un résultat.

Le cas qui vous concerne

Sur les variables de revenu et de patrimoine, la non-réponse est presque toujours du troisième type.

- Les très hauts revenus déclarent moins souvent.
- Les revenus non déclarés sont donc, en moyenne, plus élevés.
- Supprimer ces lignes sous-estime la moyenne et les inégalités.

On ne peut pas trancher à partir des données seules : par construction, on ne voit pas ce qui manque.

Ce qu'on peut faire, c'est **le dire**, et mesurer la sensibilité du résultat à l'hypothèse retenue.

Hors champ n'est pas non-réponse

Trois colonnes du fichier contiennent des cases vides. Aucune des trois ne signifie la même chose.

Colonne	Effectif	Ce que le vide signifie
SALAIRE_NET = 0	675	Non-salarié : la question ne se pose pas
MOTIF_ABS vide	7 153	N'a pas été absent : sans objet
PRIME vide	4 442	N'a touché aucune prime

Une personne **hors champ** n'aurait pas dû répondre. Une **non-réponse** aurait dû. Les traiter de la même façon est une faute.

Remplacer une prime absente par la prime moyenne invente un versement qui n'a pas eu lieu. Supprimer la ligne élimine précisément les salariés sans prime, c'est-à-dire la majorité.

Décider

Nous savons maintenant **trouver** les manquants et dire **d'où ils viennent**.

- Sur le salaire, la non-réponse croît avec le diplôme : elle n'est pas due au hasard.
- Un vide n'est pas toujours un manquant : la prime vide d'un salarié sans prime est une *information*.

Reste à **décider quoi faire**. C'est l'objet de cette dernière partie — et vous rencontrerez en TD les deux pièges que nous n'avons pas le temps de voir ici : le code 9 qui n'est pas une non-réponse, et la censure haute.

Trois options, aucune neutre

Option	Ce qu'on fait	Ce que cela coûte
Supprimer les lignes	on retire les observations incomplètes	échantillon réduit, biais possible
Imputer	on remplace par une valeur	fausse précision
Conserver et signaler	on ne touche à rien, on rapporte les effectifs	analyse plus lourde

Signaler n'est pas une action sur les données : c'est une action sur le compte rendu. On laisse les manquants en place, les calculs les ignorent, et on dit combien d'observations ont réellement servi.

C'est le cas général — et la seule option qui ne suppose rien. Supprimer suppose que les manquants sont sans structure ; imputer suppose qu'on sait par quoi les remplacer.

Supprimer : ce qui disparaît vraiment

```
avant = len(com)
propre = com.dropna(subset=["TAUX_CHOMAGE", "NB_ENTREPRISES"])
print(avant, "->", len(propre))           # 45 -> 37
```

Huit communes sur quarante-cinq, soit 18 % du fichier. Le vrai problème n'est pas le nombre :

	Conservées	Supprimées
Population moyenne	156 380	1 786
Communes rurales	0	6

Le contrôle à faire

Comparez les lignes supprimées aux lignes conservées. Ici la catégorie rurale disparaît entièrement : le fichier nettoyé ne décrit plus les communes françaises, mais celles de plus de trois mille habitants.

Imputer par la moyenne : ce que cela casse

```
df["SALAIRE_NET"].fillna(df["SALAIRE_NET"].mean())
```

Sur les 7 326 postes salariés, dont 524 sans salaire renseigné :

	Conserver	Imputer
Moyenne	2 610	2 610
Écart-type	1 465	1 412
Corrélation avec le diplôme	0,356	0,342
Observations annoncées	6 802	7 326

La moyenne ne bouge pas d'un centime. C'est ce qui rend l'opération dangereuse : rien ne signale que quelque chose a changé.

Ce qui change, c'est la dispersion, les corrélations, et le nombre d'observations annoncé. Les erreurs-types seront trop petites, et l'inférence trop confiante.

Modifier un tableau : toujours .loc

Pour mettre à jour certaines lignes, l'écriture intuitive ne fonctionne pas :

```
df[df["SALAIRE_NET"] == 999999]["SALAIRE_NET"] = np.nan
```

Deux crochets successifs : le premier fabrique une **copie** des lignes filtrées, le second modifie cette copie — qui disparaît aussitôt. **df n'a pas changé**. Pandas affiche au mieux un avertissement.

```
df.loc[df["SALAIRE_NET"] == 999999, "SALAIRE_NET"] = np.nan
```

.loc désigne lignes et colonnes en une seule opération, directement dans df. **Pour modifier, c'est toujours .loc**. Pour simplement regarder, les doubles crochets suffisent.

Écrire votre première fonction

Quatre colonnes de ce fichier codent la non-réponse par un nombre. La règle s'écrit une fois.

```
def nettoyer(serie, codes_manquants):  
    s = pd.to_numeric(serie, errors="coerce")  
    return s.replace(codes_manquants, np.nan)  
  
df["SALAIRE_NET"] = nettoyer(df["SALAIRE_NET"], [999999])  
df["PRIME"]       = nettoyer(df["PRIME"],       [999999])  
df["HEURES"]      = nettoyer(df["HEURES"],      [99])  
df["DIPLOME"]     = nettoyer(df["DIPLOME"],     [9])
```

Trois bénéfices : la règle est écrite une seule fois, elle porte un nom qui dit ce qu'elle fait, et la corriger la corrige partout.

Notez la ligne absente : MOTIF_ABS. Vous découvrirez en TD pourquoi on ne peut pas lui appliquer le code 9. La liste de codes ne vient jamais d'une habitude : elle vient du dictionnaire, colonne par colonne.

Documenter

Le journal de nettoyage

Étape	Décision	Lignes	Salaire moyen	Justification
Chargement	—	8 012	67 745	Fichier brut
Code 999 999 → manquant	Conversion	8 012	2 374	Dictionnaire, SALAIRE_NET
Non-salariés	Exclusion	7 337	2 610	Hors champ : pas de salaire

Deux observations sur ce tableau.

- Traiter un seul code de non-réponse fait passer la moyenne de 67 745 à 2 374. Une ligne de code, un facteur vingt-huit.
- Retirer les non-salariés la fait remonter à 2 610, parce que leurs zéros tiraient la moyenne vers le bas.

Une décision non documentée est une décision cachée.

Ce tableau suppose une règle : les données brutes ne se modifient jamais. On charge dans `brut`, on travaille sur `df = brut.copy()`.

Pour la suite

TD 3 — Nettoyer un vrai fichier

- Repérer les non-réponses codées en chiffres, et les convertir
- Distinguer non-réponse et hors champ — deux choses différentes
- Identifier une censure haute et dire ce qu'elle fait à un indice d'inégalité
- Documenter chaque décision, et mesurer son effet sur le résultat

Le livrable n'est pas un fichier propre : c'est le **journal des décisions**, avec l'effet chiffré de chacune.

`stefaniamarcassa.github.io/analyse_des_donnees`

Des questions ?